



The Oracle Asymmetry

Where evolutionary self-improvement of an agent gets measurable traction

Date: June 15, 2026

Organization: Agent Self-Evolution

Repository: github.com/jramos/agent-self-evolution

Abstract

A framework for evolving an agent's skills, tool descriptions, prompts, and code was exercised across many experiments. The consolidated result is a **spend-allocation** finding shaped like an asymmetry. Self-evolution produced deploy-grade traction under a **conjunction** of conditions — a ground-truth executable oracle, real headroom, and a code repair driven by failing-test feedback. On real historical bugs this produced a gate-passing, oracle-matching fix for **12 of 20** bugs (deploy-reachable **0.60**, Wilson 95% CI [0.39, 0.78]; it clears a pre-registered **futility floor** of 0.10, which certifies "not catastrophic," not a validated deployment rate). A leakage check shows the test's expected values are load-bearing — withhold them and 11/12 of the successes fall to 3/12 — so this is **test-feedback repair, not autonomous re-derivation**. Where the fitness signal is instead an LLM judge or a capable agent's behavior, we measured **no detectable effect** — but at a power that resolves only *large* couplings (see below). The two arms are instruments of unequal resolution, and oracle-presence is confounded with headroom and task type, so this is a direction-setting result — not a proof of inertness, and not a single "oracle vs. agent" law.

1. The positive result: executable-oracle test-feedback repair

Real historical tool bugs were harvested from an active repository's git stream, each carrying its upstream fix commit as a **ground-truth oracle**. The loop repairs the buggy parent in an isolated worktree from failing-test feedback; a repair counts only if it passes the full fix-commit test set and behaviorally matches the upstream fix. The organism (one bug) is the unit, and a bug is deploy-reachable when a majority of 3 independent seeds produce an oracle-matching fix.

Estimand (organism-level)	Value
Deploy-reachable	12 / 20 = 0.60
Wilson 95% CI	[0.387, 0.781]
Cluster bootstrap (mean, CI)	0.602, [0.40, 0.80]
Clears futility floor (0.10)?	yes - P(below) = 0.0
ICC / design effect / effective-N	0.326 / 1.65 / 36.3
Cost	\$3.10

Read the verdict honestly. The 0.10 line is a futility floor, not a success bar — deploy-reachable would have to collapse to 4/20 before it failed. The honest headline is the point estimate **0.60 [0.39, 0.78] on one repository, one proposer model**: promising, not settled. The number is also fragile at the knee: the per-seed distribution is bimodal (0 correct: 1 organism, 1: 7, 2: 2, 3: 10), so 10 organisms are unanimous successes, 8 are clear failures, and **0.60 hinges on the 2 organisms sitting exactly at 2/3** — each one seed-flip from flipping the verdict. The cluster bootstrap resamples the already-dichotomized flags, so [0.40, 0.80] does not propagate the

3-seed noise on each organism; a two-stage bootstrap would be wider. The pooled per-seed rate ($41/60 = 0.68$) is recorded only for contrast and labelled dishonest in the source artifact (ICC 0.33 inflates its precision); 0.60 is a different, threshold-dependent estimand, not a de-biased 0.68.

Leakage check — the expected values are load-bearing (run). The repair loop feeds the proposer the failing test's full output, which for assertion failures includes the expected values, and the gate verifies an oracle match with the held-out split off. We re-ran the 12 deploy-reachable organisms through the same gate under two feedback regimes: a full-rollback control, and a no-leak arm (`--tb=no` plus the failing node-ids, so the proposer learns which tests fail but not their expected values).

Feedback regime	Deploy-reachable (of 12)
Full traceback (control)	11 / 12 - reproduces the headline
No-leak (expected values withheld)	3 / 12

Withholding the expected values collapses the rate from ~0.92 to 0.25 on the very organisms that succeeded. So **0.60 is test-feedback repair leaning on the test's expected values, not autonomous re-derivation** — the loop reads the failing test (as a developer would) and the concrete assertions do ~2/3 of the work. In production the failing test is always present, so 0.60 is the deployable test-feedback-repair rate; the **3/12 (~0.25)** that survive value-withholding is the autonomy floor. Whether the answer-informed fixes generalize or hard-code was then probed with a fuzzed differential: fresh inputs (never in the tests) through each control fix and the upstream oracle, compared. Of the 11 control successes, **4 are meaningfully testable** (distinct non-trivial returns: `patch_parser` across 7 diverse patches, `discord_tool`, `file_tools`, `kanban_tools`) and **all 4 match the oracle on every fresh input (0 divergences)** — evidence the fixes generalize rather than hard-code. Thin, though: 3 more return constants (match only confirms "doesn't raise"), and 4 need fixtures to call. Of 33 harvested candidates: 20 valid organisms, 7 source-missing, 5 not-valid, 1 too-large.

2. The null: capable-agent artifact decoupling

The sharp, defensible form of this result is about **inferability**, not behavior in general: when a capable agent (gpt-5-mini class) can recover a tool's behavior from its **name plus the task**, the description text is redundant, and editing it — better or worse — does not move behavior.

- **Improving did not help.** Evolving skills, tool descriptions, and turn-level sequences produced null deploy results, including under a judge-free test-command verdict (so judge saturation is not the cause). These baselines were also largely already-saturated / hand-tuned — low headroom, so little to gain.
- **Degrading did not hurt.** A strongly misdirecting description, and even an actively false one ("merges and preserves content, never deletes"), each produced 0/7 caught-losses against a 0/7 A/A false-alarm floor. The probe tool's name states its function — the weakest possible test of description sensitivity, which is exactly why the agent routes past

the text.

Power is the binding caveat. With $n=7$ per arm, 0/7-vs-0/7 resolves only large couplings: a one-sided Fisher test cannot separate treatment from the floor until the caught-loss rate reaches about 4/7 (~57%); at 80% power the minimum detectable effect is roughly 40%. So "no detectable effect" means we can rule out only effects larger than about half — not "we had good power and saw nothing." A delivery canary (a description that changes which file is written) separates 1.00 vs 0.00, proving the harness records routing; it does not prove the harness could detect a realistic, intermediate-magnitude quality effect — that control was not run. This is a failure to reject the null on one capable-agent class, for name-inferable tools, on low-headroom artifacts; it bounds the effect and predicts the coupling reappears where the agent cannot infer behavior from priors.

3. The dead ends: two honestly-distinct kinds

Supply-absent (a real boundary). Over a year of one active repo's history there were zero clean, tool-local dependency-version regressions: manifest-touching commits were broad migrations or feature additions. This delimits regardless of effect size — the instances do not exist in the corpus.

Underpowered nulls (not boundaries). These found no usable signal at small N , but their intervals are wide — they steer spend, they do not prove impossibility:

- **Metamorphic verification (no oracle): 0/8.** Generic invariants caught 0 of 8 real bugs (Wilson upper ~0.37, consistent with metamorphic working ~1-in-4). Real tool bugs are input-specific edge cases that do not violate broad invariants.
- **Held-out-split independence: 3/8.** Only 3 of 8 bugs had tests input-diverse enough for an independent held-out, so the independent held-out product is deferred — the oracle sidesteps it for measurement.

4. The unifying thesis, and its limit

Across the classes, traction tracked **how concrete and mechanical the verdict's signal is**: a deterministic test with literal expected values yielded a gradient; a judge or a capable agent's behavior did not. The leakage check makes the mechanism explicit — the gradient is the test's expected values, exactly what the behavioral arm lacks — so the asymmetry is sharpened. But it is still a contrast between two instruments of unequal resolution (one resolves effects of 0.10, the other only ~0.5), and oracle-presence is confounded with headroom and task type, so the clean "oracle vs. agent" axis is a hypothesis this campaign suggests but did not isolate. The disconfirming experiment never run: apply the loop to an already-correct function (test present, no headroom) — if the gradient vanishes, headroom, not the test, was load-bearing. Practically, the implication stands: spend budget where a concrete executable test and real headroom exist (code repair, fed the failing test by the shipped propose-only triage sentinel — the production regime where 0.60 holds), and not on artifact quality for capable, name-inferable surfaces.

5. Limitations

- **Test-feedback, not autonomous re-derivation (checked).** The leakage check showed the test's expected values are load-bearing: withholding them drops 11/12 to 3/12. So 0.60 is the test-feedback-repair rate (the production regime), not autonomous re-derivation (~ 0.25). A fuzzed differential cleared the 4 control fixes it could meaningfully test (0 divergences from the oracle on fresh inputs) — evidence against hard-coding, but thin (4 of 20).
- **Underpowered null.** $n=7$ resolves only effects above $\sim 50\%$; the null is an upper bound on a large effect, scoped to one capable-agent class and name-inferable tools.
- **Confounded axis.** Oracle-presence co-varies with headroom and task type; the one-axis thesis is suggested, not isolated.
- **ICC instability.** ICC swung 0.33 vs 0.57 across runs; effective- N (36.3) and the verdict's precision rest on a noisy estimate.
- **One corpus, one proposer.** $N=20$ on one repository's bug stream with one mini-class model; the dead-end pilots (metamorphic 0/8, held-out 3/8) are small- N with wide CIs.